

```

#include <iostream>
using namespace std;

const int MAX = 100;
int visited[MAX];

class Queue {
    int arr[MAX];
    int front, rear;
public:
    Queue() { front = rear = -1; }
    bool isEmpty() { return front == -1; }
    bool isFull() { return (rear + 1) % MAX == front; }
    void enqueue(int val) {
        if (isFull()) { cout << "Queue is full\n"; return; }
        if (isEmpty()) front = rear = 0;
        else rear = (rear + 1) % MAX;
        arr[rear] = val;
    }
    int dequeue() {
        if (isEmpty()) return -1;
        int val = arr[front];
        if (front == rear) front = rear = -1;
        else front = (front + 1) % MAX;
        return val;
    }
};

// BFS from a single starting node
void BFS(int adj[MAX][MAX], int n, int SN) {
    Queue q;
    q.enqueue(SN);
    visited[SN] = 1;
    while (!q.isEmpty()) {
        int cur = q.dequeue();
        cout << cur + 1 << " ";
        for (int i = 0; i < n; i++) {
            if (adj[cur][i] == 1 && visited[i] != 1) {
                q.enqueue(i);
                visited[i] = 1;
            }
        }
    }
}

```

```

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;
    cout << "Enter edges (source destination):\n";
    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == 0) {
            adj[u - 1][v - 1] = 1;
            // Undirected? also: adj[v-1][u-1] = 1;
        } else {
            cout << "Invalid edge\n";
            i--;
        }
    }
}

int main() {
    int n, e;
    int adj[MAX][MAX];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdgesToAdjMatrix(adj, n, e);
    int start;
    cout << "Enter starting node: ";
    cin >> start;
    for (int i = 0; i < n; i++) visited[i] = 0;
    cout << "\nBFS Traversal: ";
    BFS(adj, n, start - 1);
    // Handle disconnected components
    for (int i = 0; i < n; i++) {
        if (visited[i] != 1) {
            cout << "\n(disconnected, new BFS from " << i + 1 << ") ";
            BFS(adj, n, i);
        }
    }
    cout << endl;
    return 0;
}

```

WHEN TO USE BFS (Scenario Triggers)

- ▶ Question asks for the SHORTEST path in an unweighted graph
- ▶ Question mentions "level by level" or "minimum number of steps/hops"
- ▶ Finding shortest path in a maze / grid where every move costs the same
- ▶ Checking if a graph is bipartite (2-coloring using BFS layers)
- ▶ Finding all nodes within K steps/hops of a source
- ▶ Graph is DENSE (many edges) — class notes explicitly say prefer BFS here
- ▶ Social network "degrees of separation" style questions

BFS vs DFS — Which One?

- ▶ Need SHORTEST path / fewest edges? → BFS (guaranteed shortest in unweighted graph)
- ▶ Need to explore full depth, detect cycles, topological order? → DFS instead
- ▶ Graph is wide/dense? → BFS. Graph is deep/narrow? → DFS uses less memory
- ▶ BFS uses a QUEUE (FIFO). DFS uses a STACK (LIFO) — this is the core mechanical difference

MANUAL TRACE METHOD (How to Solve by Hand)

Use this exact process if the exam gives you a graph and asks you to trace BFS:

- ▶ 1. Pick the start node. Mark it visited. Enqueue it. Write it in the Output list.
- ▶ 2. Dequeue the front node from the queue.
- ▶ 3. Look at ALL its neighbours in order (usually smallest index / alphabetical first).
- ▶ 4. For each UNVISITED neighbour: mark visited, add to Output, enqueue it.
- ▶ 5. Repeat step 2-4 until the queue is empty.
- ▶ 6. If nodes remain unvisited, graph is disconnected — restart BFS from any unvisited node.

Keep a Queue column and a Visited row/table as you go (matches the class notes format) — write the queue's contents after each dequeue/enqueue step, this is what the exam checks.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Disconnected graphs — same rule as DFS: if not all nodes visited from one start, restart BFS from an unvisited node.

Queue vs Stack mix-up — a very common exam trap is writing DFS logic (stack/LIFO) but calling it BFS. Double-check you enqueue ALL unvisited neighbours before moving to the next node, not just one.

1-indexed input, 0-indexed storage — same off-by-one risk as DFS. Node 1 the user types = index 0 internally.

BFS Tree — many exam questions ask you to DRAW the BFS tree, not just list the output order. The BFS tree shows parent-child edges only (the edge that FIRST discovered each node), not all original graph edges.

"Visited" marked at enqueue time, not dequeue time — mark visited the moment you enqueue a node, otherwise the same node can be added to the queue multiple times.

EXAMPLE 1 — Small Connected Graph (Trace by Hand)

Graph (undirected): A-B, A-C, B-D, B-E, C-F, E-F

Start node: A

Step-by-step trace:

- ▶ Queue: [A] → Visited: {A}
- ▶ Dequeue A → neighbours B, C (both unvisited) → enqueue both
- ▶ Queue: [B, C] → Visited: {A, B, C}
- ▶ Dequeue B → neighbours A(visited), D, E → enqueue D, E
- ▶ Queue: [C, D, E] → Visited: {A, B, C, D, E}
- ▶ Dequeue C → neighbours A(visited), F → enqueue F
- ▶ Queue: [D, E, F] → Visited: {A, B, C, D, E, F}
- ▶ Dequeue D → no unvisited neighbours
- ▶ Dequeue E → neighbours B(visited), F(visited) → nothing new
- ▶ Dequeue F → neighbours C(visited), E(visited) → nothing new
- ▶ Queue empty → done

Output order: A, B, C, D, E, F

BFS Tree (parent-child edges only): A→B, A→C, B→D, B→E, C→F

EXAMPLE 2 — Shortest Path Scenario (CP-style)

Scenario: "Find the minimum number of bus transfers to travel from Station A to Station E, given a network of direct bus routes between stations."

How to apply BFS: treat each station as a node, each direct route as an unweighted edge. Run BFS from A. The FIRST time you reach E, the number of steps taken (BFS layer/depth) is the minimum number of transfers. Track this using a `dist[]` array where `dist[start]=0` and `dist[neighbour]=dist[current]+1`.

EXAMPLE 3 — Disconnected Graph (Restart Scenario)

Graph: A-B, C-D (two separate components)

Start node: A → BFS visits only A, B. Nodes C, D remain unvisited.

Exam expects you to say: "Since C is unvisited after the first BFS call, restart BFS from C." Final Output: A, B, C, D (two separate BFS calls combined).

EXAMPLE 4 — Bipartite Check Twist (Medium/Hard style)

Scenario: "A teacher wants to split students into two debate teams such that no two friends are on the same team."

How to apply BFS: model students as nodes, friendships as edges. Run BFS, coloring each node with an alternating color (0/1) from its parent. If you ever find an edge connecting two same-colored nodes, it's impossible (graph is not bipartite) — meaning the friend groups cannot be split into two teams cleanly.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER**Before you finalize any BFS answer, check:**

- Did I mark nodes visited at ENQUEUE time, not dequeue time?
- Did I process ALL neighbours of a node before moving to the next queue item?
- Did I handle disconnected components with a restart?
- If asked for BFS Tree, did I only draw parent-child (discovery) edges, not all graph edges?
- Did I follow the correct neighbour order (usually smallest index / alphabetical first)?